

BioParcela

Manual Tecnico de Funcionalidades

Juan López Sarrelangue

28 de octubre de 2026

Módulos administrativos BioParcela

Alcance: este manual cubre el frontend del módulo pecuario y sus contratos HTTP observables. El repositorio no contiene implementación de API, migraciones ni modelos físicos de base de datos.

1. Descripción general

BioParcela es un panel administrativo para consultar indicadores agrícolas y pecuarios. El módulo Resumen Pecuario presenta una vista consolidada de actividad ganadera bovina registrada, con indicadores, gráficos, filtros geográficos y exportaciones.

La pantalla se monta en el componente Pecuario y se accede por la ruta /admin/resumen-pecuario dentro del LayoutComponent. La vista es de consulta: no crea ni edita registros pecuarios desde este componente.

- Indicadores principales: productores, UPP, razas registradas e índice de trazabilidad.
- Perfiles demográficos: sexo, edad y pertenencia a pueblos originarios.
- Inventario: cabezas, vientres, finalidad zootécnica y razas.
- Unidades de producción: distribución territorial y tipo de persona.
- Gestión sanitaria, propiedad privada/comunal, organismos ganaderos y PGN.

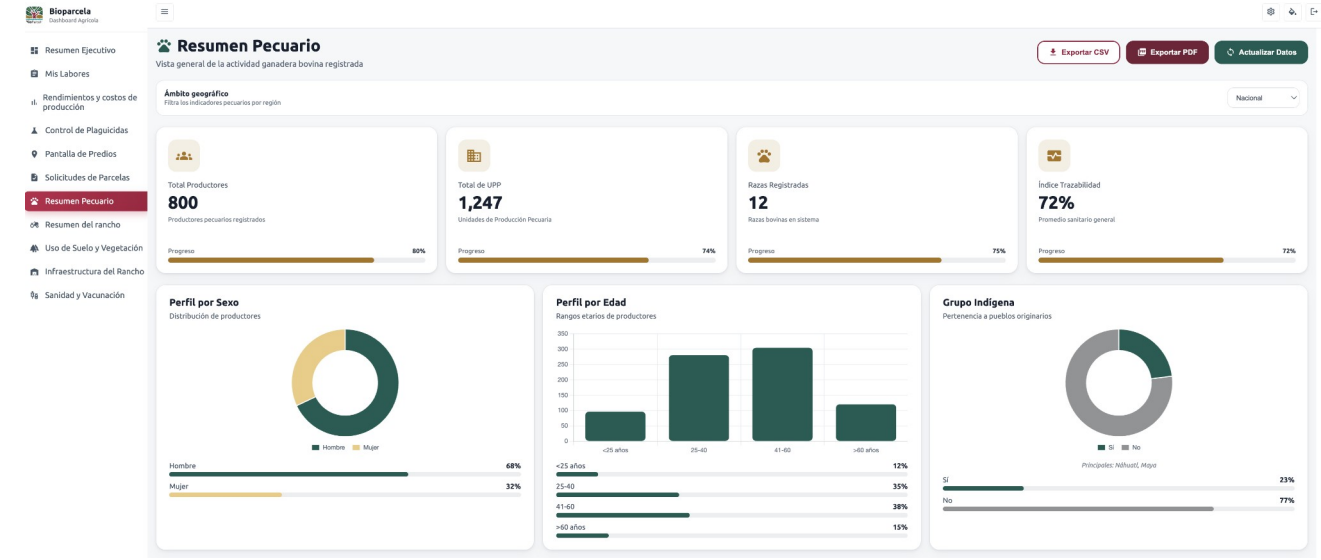


Figura 1. Vista consolidada de KPIs y perfiles del módulo pecuario

2. Tecnologías utilizadas

Tecnología	Versión / uso
Angular	21.2.x; componentes, router, SSR/hydration y compilación.
TypeScript	~5.9.2; tipado de respuestas y lógica de presentación.
Angular Material/CDK	21.2.x; iconos, layout y controles visuales.
Chart.js + ng2-charts	4.5.1 / 10.0.0; gráficas doughnut y barras.
RxJS	~7.8.0; forkJoin, catchError y finalize.
html2canvas + jsPDF	1.4.1 / 4.2.1; captura DOM y PDF A4 horizontal.

Capacitor / Electron	8.3.x / 30.x; empaquetado móvil y escritorio.
SweetAlert2 / CryptoJS	Alertas y utilidades de almacenamiento/sesión.

Las versiones se tomaron de package.json. No se documentan versiones de backend porque ese código no está en este repositorio.

3. Organización del código

```
src/app/pages/Pecuario/pecuario.component.ts      Lógica, contratos y carga de datos
src/app/pages/Pecuario/pecuario.html              Estructura de la pantalla y bindings
src/app/pages/Pecuario/pecuario.scss              Layout, colores y responsive
src/app/app.routes.ts                             Ruta /admin/resumen-pecuario
src/app/services/generic.service.ts               HTTP y Authorization Bearer src/app/services/pdf-
export.service.ts                                 Captura HTML y composición PDF
src/app/guards/auth.guard.ts                      Validación de sesión
src/environments/environment.ts                   URLs base y paths de API
```

Pecuario usa ChangeDetectionStrategy.OnPush. Los datos llegan como respuestas HTTP y se transforman en ChartData de Chart.js. La plantilla usa @if y @for para mostrar estados, listas y paneles.

Capa	Responsabilidad
Routing	Carga LayoutComponent y presenta Pecuario como hijo de /admin.
Componente	Coordina filtros, peticiones, estado de carga y exportaciones.
GenericService	Centraliza GET/POST y agrega Bearer si la petición es segura.
PdfExportService	Oculta controles, captura DOM y divide el resultado en páginas.
Chart.js	Dibuja barras horizontales/verticales y gráficas de dona.

4. Funcionalidades

4.1 Ciclo de carga

ngOnInit obtiene el año desde ActivatedRoute, carga estados y llama loadAllData. loadAllData construye parámetros geográficos y ejecuta diez observables con forkJoin. Cada observable usa catchError(() => of(null)); por ello una falla parcial no cancela el resto. finalize libera loading, marca el componente para revisión y actualiza las gráficas.

4.2 Filtros geográficos

El nivel puede ser nacional, estado o municipio. Al cambiar nivel se limpian estado y municipio. Al elegir estado en modo municipio se consulta el catálogo de municipios y se normalizan códigos a dos dígitos para estado y tres para municipio.

4.3 Actualización

Actualizar Datos ejecuta POST /api/pecuario/cache/flush con cuerpo vacío. Al finalizar, incluso si el flush falla, el componente vuelve a cargar los datos. La interfaz deshabilita el botón mientras refreshing o loading.

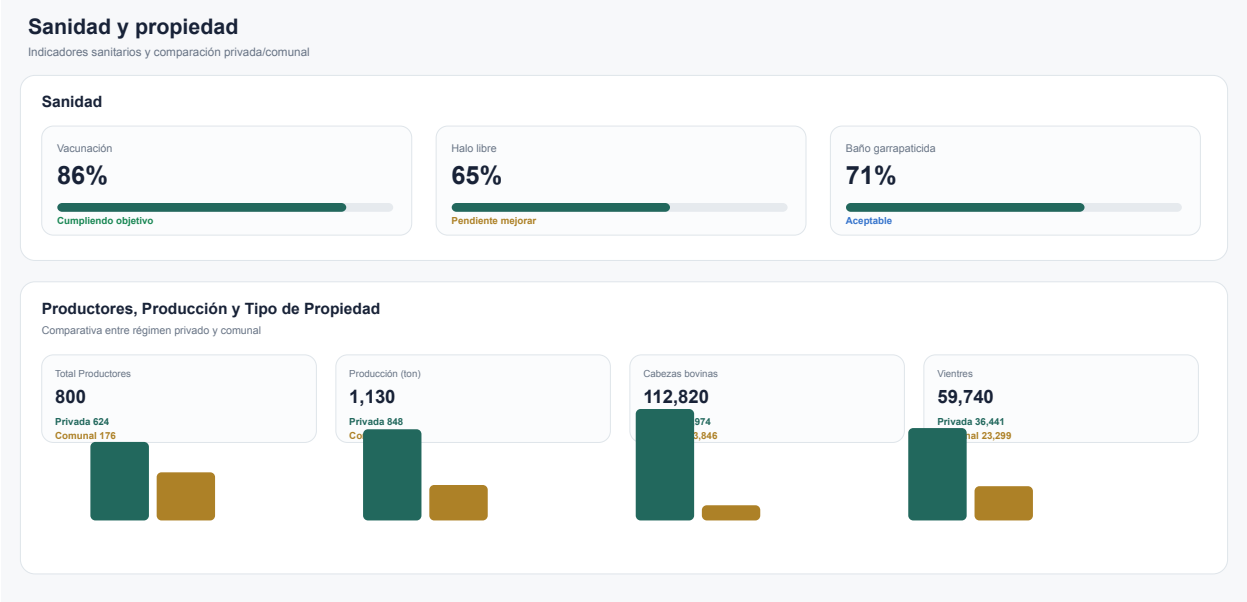


Figura 2. Paneles de Sanidad y Propiedad; ilustran la lectura visual de indicadores porcentuales y comparativas.

5. Autenticación

Login valida usuario y contraseña no vacíos, agrega información del navegador/dispositivo y envía POST a /api/v1/login. La respuesta esperada contiene token. El token se guarda en sessionStorage con la clave userSessionAdmonOS y luego se navega a /admin.

GenericService recupera la sesión desde sessionStorage o localStorage. En llamadas marcadas isSecure=true agrega Authorization: Bearer <token>. Las llamadas del módulo pecuario usan isSecure=true.

AuthGuard existe y redirige a /login cuando no hay sesión. Comportamiento actual importante: canActivate está comentado en la ruta admin dentro de src/app/app.routes.ts. Por tanto, el frontend no está imponiendo el guard en esa ruta; la protección efectiva debe existir también en API y debe revisarse antes de producción.

```
POST https://giz-qa.encodingmx.com/api/v1/login Body: { access, password, device } Respuesta mínima: { token } Sesión: userSessionAdmonOS
```

6. Visualización de datos

Las gráficas se crean después de que afterNextRender marca chartsReady. Las gráficas de dona usan colores verde/dorado/gris. Las gráficas de barras usan ejes configurados desde ChartConfiguration y se redimensionan con refreshChartsSoon.

Endpoint	Método	Datos mostrados	Parámetros
/api/pecuario/resumen	GET	KPIs, sexo, edad, indígena	Sin parámetros en frontend actual
/api/pecuario/cabezas-vientres	GET	Finalidad, cabezas y vientres	nivel, estado, municipio
/api/pecuario/razas	GET	Razas y cantidades	nivel, estado, municipio

/api/pecuario/upp	GET	Total y desglose de UPP	nivel, estado, municipio
/api/pecuario/upp/tipo-persona	GET	Persona física/moral	nivel, estado, municipio
/api/pecuario/trazabilidad	GET	Guías REEMO y detalle	nivel, estado, municipio
/api/pecuario/sanidad	GET	Vacunación, halo, baño e índice	nivel, estado, municipio
/api/pecuario/propiedad	GET	Privada/comunal	nivel, estado, municipio
/api/pecuario/organismos-ganaderos	GET	Organismos y enlace PGN	nivel, estado, municipio
/api/pecuario/comparacion-pgn	GET	Serie mensual BioParcela/PGN	anio

Exportar CSV llama /api/pecuario/export con los parámetros geográficos actuales y descarga el Blob como resumen-pecuario.csv. El selector Cabezas/Vientres cambia el dataset local y no requiere una nueva petición.

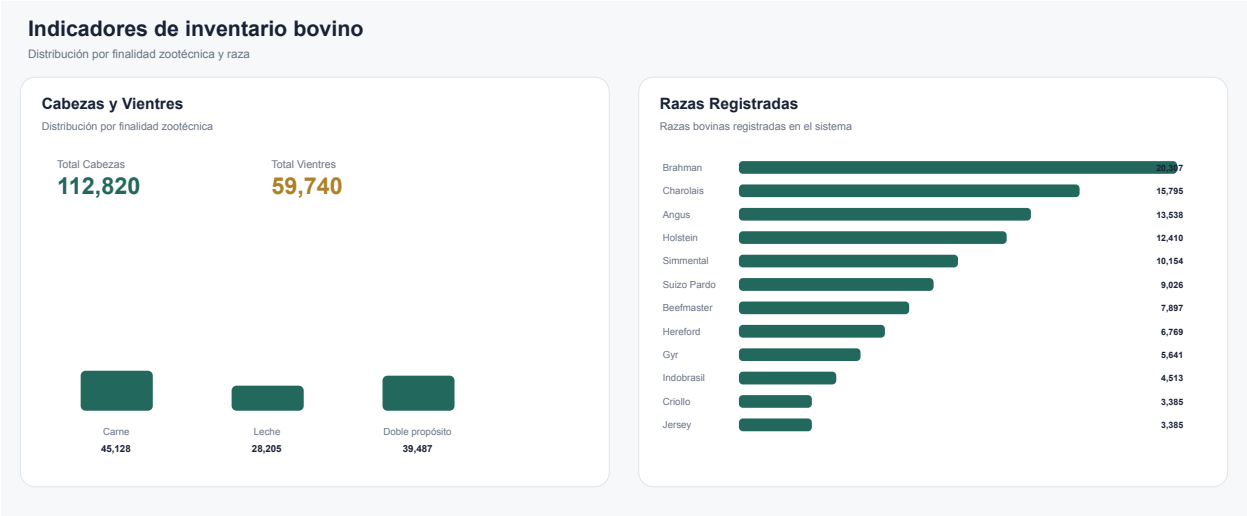


Figura 3. Indicadores de inventario: finalidad zootécnica y razas registradas.

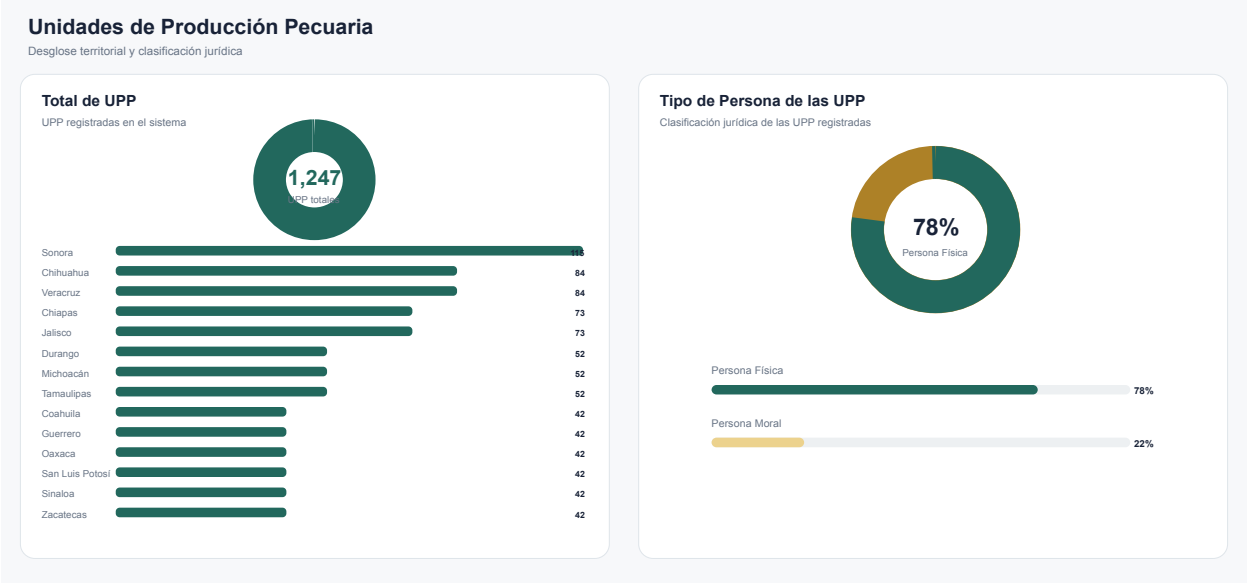


Figura 4. UPP por territorio y clasificación de persona.

7. Generación de PDF

El botón Exportar PDF envía el elemento #pdfContent a PdfExportService.exportElement. El servicio espera document.fonts.ready y un frame de renderizado para capturar las gráficas. Después oculta botones, enlaces y selectores de filtro/control mediante CSS e ignoreElements.

Captura: html2canvas con useCORS, allowTaint=false y escala máxima 2.

Formato: jsPDF A4 horizontal, 297 × 210 mm, margen de 10 mm.

Paginación: busca límites naturales en header, section, article, panel, chart-box y listas.

Salida: resumen-pecuario.pdf. La generación usa el DOM actual; no solicita un PDF al backend.

Riesgo operativo: el PDF depende de que las gráficas ya estén renderizadas y de que las imágenes externas permitan CORS. El servicio restaura visibilidad y display de controles en finally.

8. Integración con Trazabilidad

La sección Trazabilidad consume /api/pecuario/trazabilidad. Presenta el porcentaje guias_emitidas_pct y detalle por tipo de guía. statusLabel traduce estados como cumpliendo_objetivo, pendiente_mejorar, aceptable y critico.

La interconexión PGN se alimenta de /api/pecuario/organismos-ganaderos. El contrato puede entregar productores_con_perfil, total_productores y pgn_url. Comparación Bioparcela vs PGN consume /api/pecuario/comparacion-pgn con el año seleccionado y muestra bioparcela, pgn, coincidencia, cobertura_pct, sin_registro_pgn y serie_mensual.

Si pgn_url existe, la plantilla abre el destino en una pestaña nueva con rel="noopener". No hay sincronización de escritura desde el frontend; la interconexión es de consulta y enlace externo.

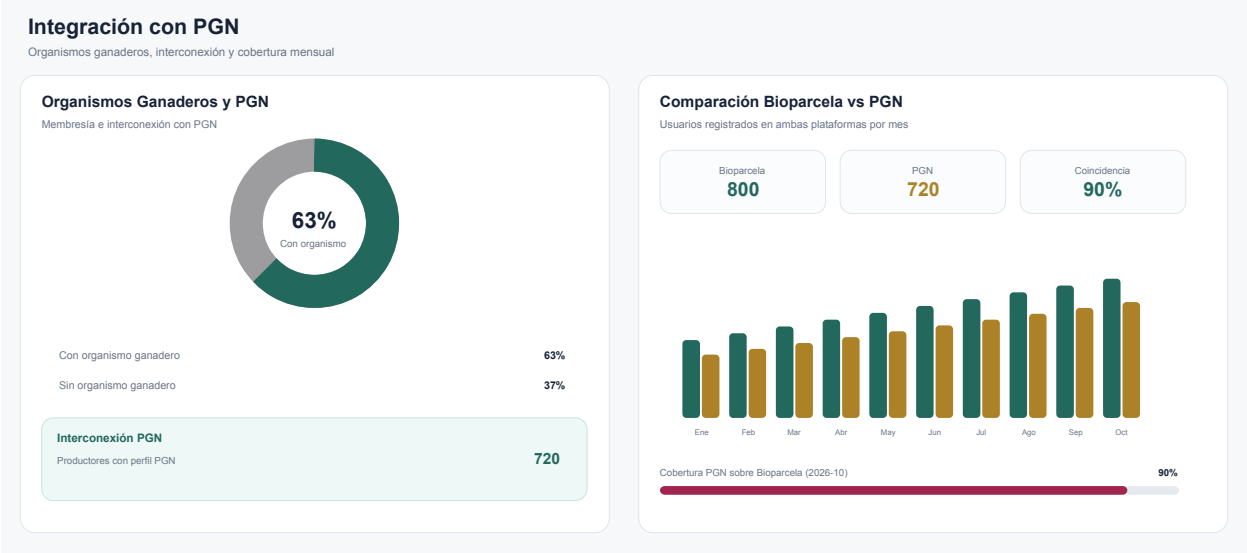


Figura 5. Organismos ganaderos, cobertura PGN y comparación mensual.

9. Base de datos

El repositorio contiene únicamente el consumidor frontend. No contiene migraciones, entidades ORM, SQL ni definición física de base de datos. La siguiente tabla documenta el modelo lógico que el frontend espera de la API; no sustituye el esquema oficial del backend.

Entidad lógica	Campos observables	Relación funcional
Productor	total_productores; sexo; edad; indigena	Se agrupa por UPP, región y perfiles demográficos.
UPP	total; desglose; tipo de persona	Pertenece al ámbito geográfico y concentra inventario.
Inventario bovino	cabezas; vientres; finalidad; raza	Se relaciona con UPP y finalidad zootécnica.
Trazabilidad / REEMO	guias_emitidas_pct; detalle	Resume movilizaciones asociadas al inventario/UPP.
Sanidad	indicadores; indice_general	Indicadores sanitarios agregados por ámbito.
Propiedad	privada; comunal; producción_ton	Compara régimen de tenencia para productores e inventario.
Organismo ganadero	con_organismo; sin_organismo; PGN	Relaciona productor BioParcela con perfil PGN.
Comparación PGN	bioparcela; pgn; cobertura; serie_mensual	Cruza usuarios activos por mes y año de corte.

Relación transversal: las consultas geográficas usan nivel nacional/estado/municipio; el frontend envía códigos normalizados. Para validar cardinalidades, índices, llaves y reglas de integridad se necesita el repositorio del backend o su contrato OpenAPI.

10. Configuración y ejecución

Elemento	Valor / instrucción
Node.js	Usar una versión compatible con Angular 21; desarrollo validado con Node 22.15.0.
npm	packageManager npm@10.9.2. Ejecutar npm install.
Desarrollo	npm start; servidor Angular en https://bioparcela.encodingmx.com.
Build	npm run build; salida dist/adminos.
Pruebas	npm test.
API	https://giz-qa.encodingmx.com/api/v1 y servicios /api/pecuario, /api/infraestructura, etc.
CORS / red	El navegador debe alcanzar la API QA y la API debe permitir el origen local.
Configuración	No hay .env en frontend; las URLs están en src/environments/environment.ts y environment.prod.ts.

```
npm install npm start # abrir https://bioparcela.encodingmx.com/admin/resumen-pecuario
```

Para probar datos reales se requiere una sesión válida y disponibilidad de los endpoints QA. En una instalación nueva, validar primero login, menú y luego carga del módulo.

11. Observaciones técnicas

La ruta admin no aplica AuthGuard porque la línea canActivate está comentada. Revisar antes de liberar.

El endpoint resumen se consume sin los parámetros geográficos que sí reciben los otros endpoints. Verificar si KPIs/perfiles deben cambiar por estado o municipio.

Cada endpoint individual tolera fallo y mantiene datos parciales; la UI solo muestra el banner general cuando ninguna respuesta carga.

El almacenamiento local/sessionStorage actualmente serializa la sesión en texto plano aunque el servicio se llama LocalStorageEncryptService. Revisar si el token debe persistir y aplicar protección XSS/CSP.

El documento cubre frontend. Confirmar esquema, permisos de API, caché y trazabilidad con el equipo backend.

12. Módulo Resumen del Rancho

El módulo Resumen del Rancho consolida indicadores de superficie, producción, apoyos, carga animal y propiedad. Se accede mediante /admin/resumen-rancho y se renderiza en el componente Rancho dentro del LayoutComponent.

- Superficie total y distribución por pastoreo, cultivo/forraje e infraestructura.
- Tamaño del productor, rangos de animales y finalidad productiva.
- Apoyos recibidos, programas, monto estimado e interconexión con SIAP.
- Tipo de producción, carga animal, capacidad estimada y uso de capacidad.
- Registro de propiedad y documentos asociados al rancho.

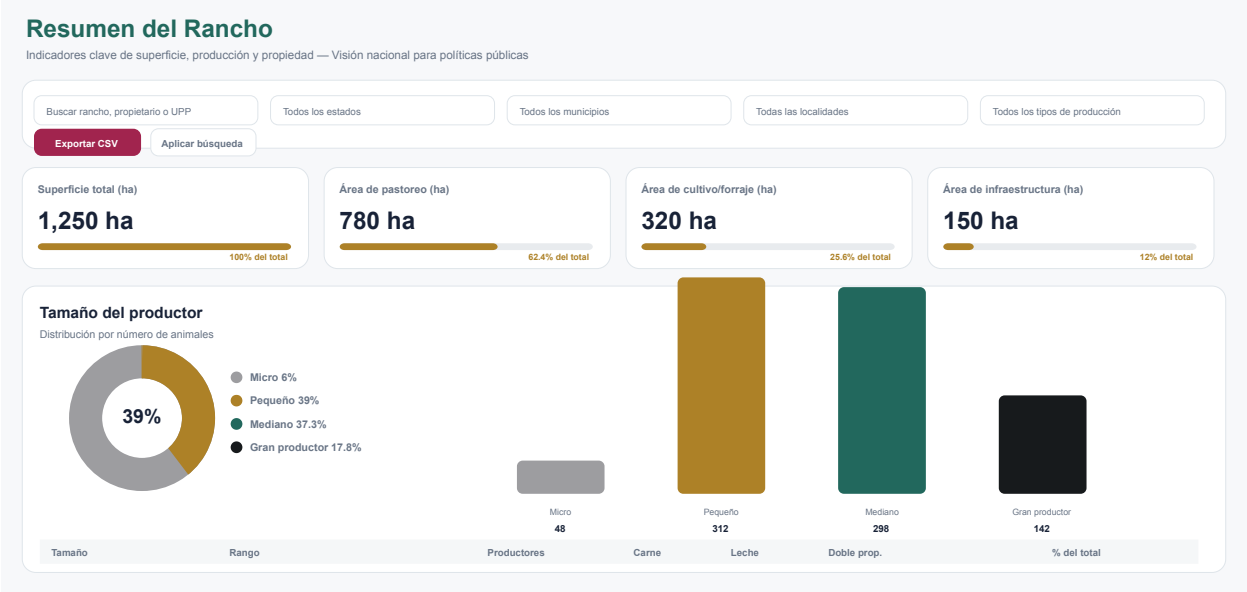


Figura 6. Filtros, KPIs de superficie y tamaño del productor. Lámina basada en la captura de referencia compartida.

12.1 Organización del código

```
src/app/pages/Rancho/rancho.component.ts   Lógica, contratos y carga de datos
src/app/pages/Rancho/rancho.html             Estructura de la vista y bindings
src/app/pages/Rancho/rancho.scss              Estilos del dashboard src/app/app.routes.ts
Ruta /admin/resumen-rancho src/environments/environment.prod.ts          URLs ranchoSuperficie,
ranchoApoyos, etc. src/app/services/generic.service.ts           HTTP y Authorization Bearer
src/app/services/pdf-export.service.ts        Exportación DOM a PDF
```

Rancho utiliza ChangeDetectionStrategy.OnPush, Chart.js/ng2-charts para gráficas y PdfExportService para exportación. La carga se coordina con forkJoin; cada respuesta se aplica solo cuando existe y el resto puede continuar si un endpoint falla.

12.2 Carga, filtros y estado

ngOnInit llama loadCatalogs y loadData. Los catálogos alimentan tipos de producción, estados, municipios y localidades. Los cambios de estado y municipio limpian filtros dependientes, recargan catálogos y vuelven a consultar los indicadores.

filterParams construye únicamente los criterios seleccionados: estado, municipio, localidad, tipo_produccion y q. La búsqueda libre se envía como q después de quitar espacios al inicio y al final. Aplicar búsqueda y cambios de filtros llaman loadData.

Control	Propiedad / parámetro	Efecto
Búsqueda	q	Filtra por rancho, propietario o UPP.
Estado	estado	Actualiza municipios y todos los indicadores.
Municipio	municipio	Actualiza localidades y todos los indicadores.
Localidad	localidad	Restringe resultados del rancho.
Tipo de producción	tipo_produccion	Restringe por actividad productiva.
Actualizar datos	POST cache/flush	Limpia caché del módulo y recarga.

12.3 Funcionalidades y visualización

Panel	Respuesta / fuente	Presentación
Superficie	superficie_total_ha, usos[]	KPI y barras porcentuales por uso.
Tamaño del productor	total_productores, tamanos[]	Dona, barras y tabla con rangos/finalidad.
Apoyos	con_apoyo, sin_apoyo, monto_total_mxn, programas[]	Tarjetas, barras de programas y monto MXN.
Tipo de producción	tipos[]	Tarjetas de porcentaje y barra segmentada.
Carga animal	cabezas, capacidad_cabezas, carga_cab_ha	Barras de carga actual/capacidad y uso.
Propiedad	propietario, rancho, empresa, ubicación, documentos[]	Ficha de propiedad y lista de documentos.

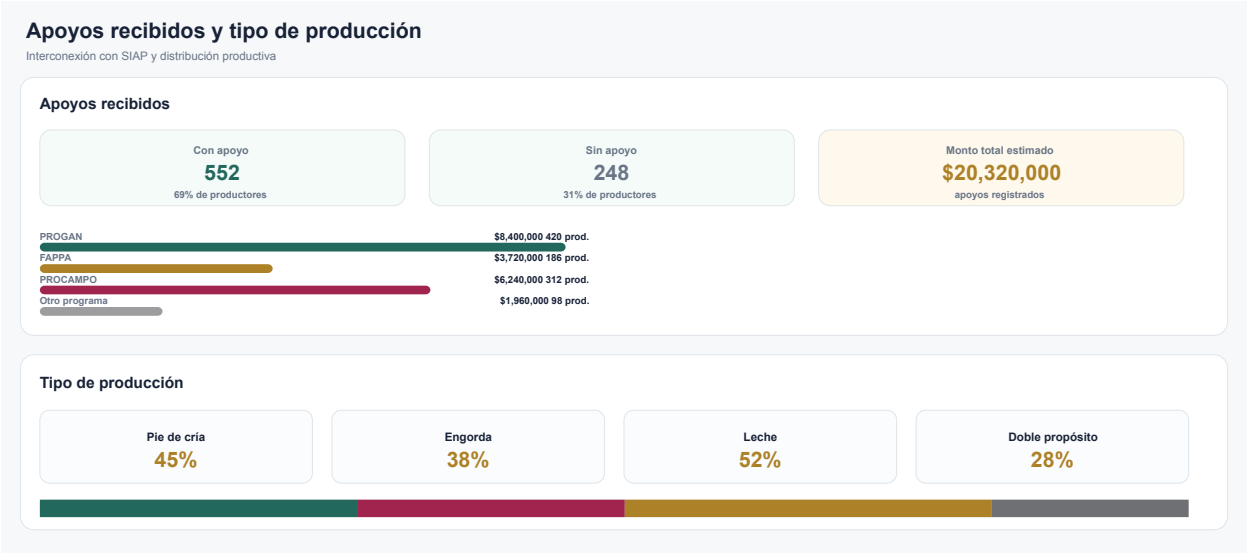


Figura 7. Apoyos recibidos, programas activos y tipo de producción.

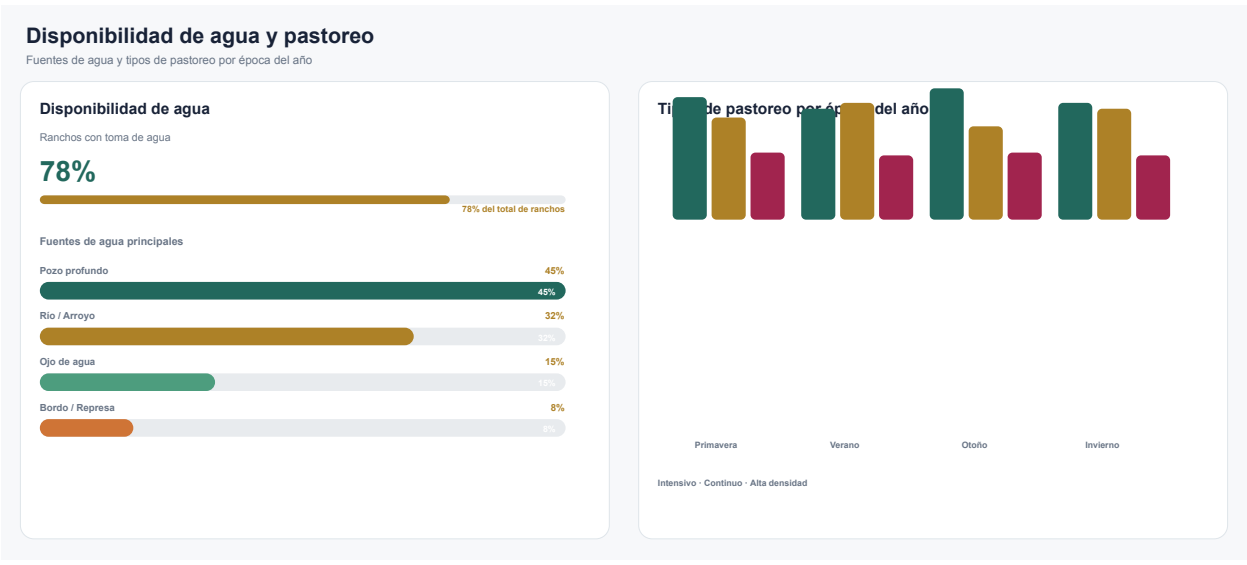


Figura 8. Carga animal, capacidad estimada y registro de propiedad.

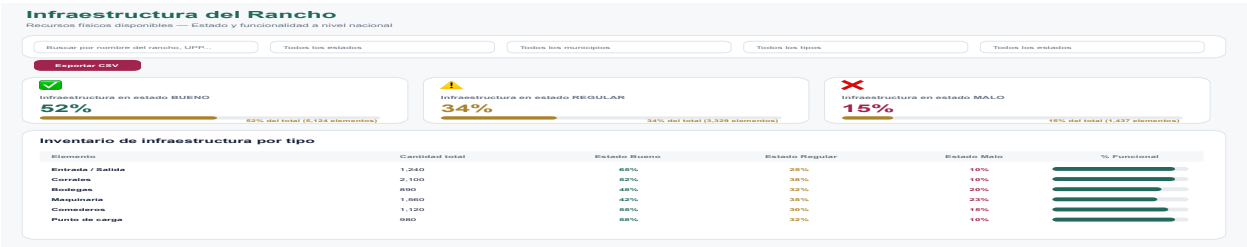


Figura 9. Distribución de superficie por uso.

12.4 Endpoints del módulo

Endpoint	Método	Uso
/api/rancho/catalogos	GET	Tipos de producción, estados, municipios y localidades.
/api/rancho/superficie	GET	Superficie total y usos.
/api/rancho/tamano-productor	GET	Tamaños, rangos, productores y finalidad.
/api/rancho/apoyos	GET	Apoyos, programas, montos y URL SIAP.
/api/rancho/tipo-produccion	GET	Distribución por tipo productivo.
/api/rancho/carga-animal	GET	Cabezas, capacidad y carga por hectárea.
/api/rancho/registro-propiedad	GET	Datos del propietario y documentos.
/api/rancho/export	GET Blob	Descarga resumen-rancho.csv con filtros actuales.
/api/rancho/cache/flush	POST	Solicita actualización de caché.

Todas las peticiones de datos y catálogo se envían con isSecure=true. Los filtros viajan como query parameters. La vista no persiste cambios de negocio; es de consulta y exportación.

12.5 Exportaciones

Exportar CSV llama ranchoExport con filterParams y descarga el Blob como resumen-rancho.csv. Exportar PDF entrega #pdfContent al mismo PdfExportService usado por el módulo pecuario; el nombre de salida es resumen-rancho.pdf.

- La exportación CSV conserva búsqueda, estado, municipio, localidad y tipo de producción.
- La exportación PDF captura los paneles visibles del estado actual.
- Los controles interactivos se ocultan durante la captura y se restauran al finalizar.

12.6 Integración con SIAP

La respuesta de apoyos puede incluir siap_url. La vista ofrece Consultar en SIAP para abrir la fuente externa y muestra una nota de interconexión con el Sistema de Información Agroalimentaria y Pesquera. El frontend no edita el padrón SIAP; consulta indicadores y ofrece el enlace de origen.

12.7 Base de datos lógica

El frontend no contiene SQL, migraciones ni entidades físicas. Los contratos permiten identificar estas entidades lógicas: Rancho/UPP, superficie por uso, productor, apoyo SIAP, tipo de producción, inventario/carga animal y documento de propiedad.

Entidad lógica	Campos observables	Relación funcional
Rancho / UPP	rancho, clave_upp, propietario, empresa, ubicación	Unidad central del resumen y filtro de consulta.
Superficie	superficie_total_ha, usos, ha, pct	Se distribuye por uso del suelo del rancho.
Productor	productores, tamanos, rango_animales	Se agrupa por tamaño y finalidad.
Apoyo SIAP	programa, monto_mxn, productores, pct_productores	Relaciona productores con programas y montos.
Carga animal	cabezas, capacidad_cabezas, carga_cab_ha	Compara inventario actual con capacidad.
Documento	nombre, extension, subido_el, url	Pertenece al registro de propiedad del rancho.

Validar llaves, cardinalidades, permisos, caché y retención de documentos con el backend. Esta sección describe el contrato observable, no el esquema físico.

12.8 Configuración y observaciones

```
npm install npm start # abrir https://bioparcela.encodingmx.com/admin/resumen-rancho
```

- Las URLs del módulo están en environment.ts y environment.prod.ts bajo ranchoPath.
- La API QA debe estar disponible y permitir CORS desde localhost:4200.
- El login y Authorization Bearer siguen el flujo documentado en la sección 5.
- El backend debe documentar el origen de datos SIAP y el manejo de documentos de propiedad.
- La ruta admin mantiene el mismo comportamiento de AuthGuard documentado previamente.

13. Módulo Uso de Suelo y Vegetación

El módulo Uso de Suelo y Vegetación monitorea cobertura vegetal, condiciones climáticas e hidrológicas, disponibilidad de agua y tipos de pastoreo. Se accede mediante /admin/uso-suelo-vegetacion y se renderiza en el componente UsoSuelo.

Cobertura: vegetación nativa, exótica, invasora e inducida.

Clima CONAGUA: precipitación, sequía, temperatura, escurrimiento y humedad/ETP.

Agua: porcentaje de ranchos con toma y fuentes principales.

Pastoreo: distribución por época del año y tipo de manejo.

Evolución histórica y alerta de tendencia de vegetación invasora.

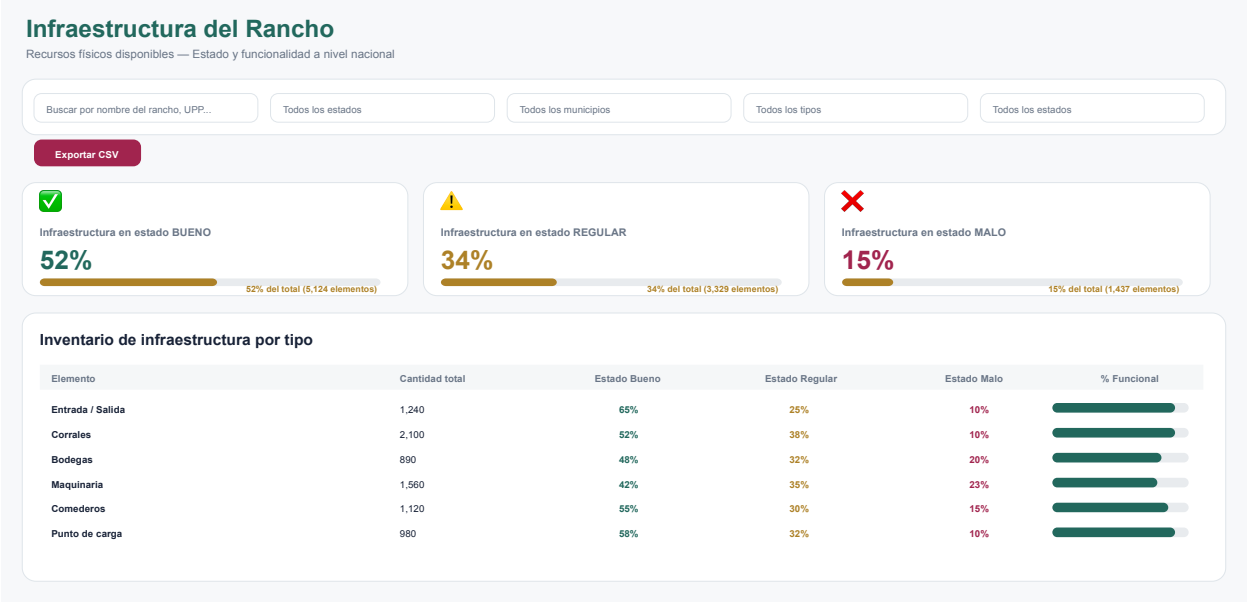


Figura 10. Filtros, cobertura vegetal y condiciones climáticas. Lámina basada en la captura de referencia compartida.

13.1 Organización del código

```
src/app/pages/UsoSuelo/uso-suelo.component.ts   Lógica, contratos y consultas
src/app/pages/UsoSuelo/uso-suelo.html             Estructura de la vista y bindings
src/app/pages/UsoSuelo/uso-suelo.scss              Estilos del dashboard src/app/app.routes.ts
Ruta /admin/uso-suelo-vegetacion src/environments/environment.prod.ts              URLs del
servicio uso-suelo src/app/services/generic.service.ts               HTTP y Authorization Bearer
src/app/services/pdf-export.service.ts             Exportación DOM a PDF
```

El componente usa ChangeDetectionStrategy.OnPush, Chart.js/ng2-charts, señales de renderizado y PdfExportService. La vista separa filtros de cobertura/temporada y controles climáticos de año, variable y periodo.

13.2 Flujo de carga y filtros

ngOnInit carga catálogos y datos. loadCatalogs obtiene estados, municipios, temporadas, años, variables climáticas y periodos. loadData construye filterParams y climateParams, ejecuta cinco peticiones con forkJoin y aplica las respuestas disponibles. Cada petición tolera error con catchError(() => of(null)).

Control	Parámetro	Comportamiento
Búsqueda	q	Busca por rancho, UPP o región.
Estado	estado	Actualiza municipios y datos del

		módulo.
Municipio	municipio	Filtra cobertura, evolución, agua y pastoreo.
Temporada / año	temporada, anio	Filtra cobertura y evolución vegetal.
Año climático	anio	Selecciona año de la consulta climática.
Variable	variable	Precipitación, sequía, temperatura, escurrimiento o humedad/ETP.
Periodo	periodo	Mensual o trimestral.

El clima recibe estado cuando existe, pero no municipio. La cobertura, evolución, agua y pastoreo reciben los filtros de temporada/año y también estado, municipio y q.

13.3 Funcionalidades y visualización

Panel	Contrato	Presentación
Cobertura	superficie_total_ha, tipos[]	Tarjetas por tipo con porcentaje y hectáreas.
Clima	fuente, conectado, actualizado_en, kpis[], serie[]	Estado de conexión, KPIs y barras Actual vs Normal histórica.
Agua	total_ranchos, con_toma_agua_pct, fuentes[]	Indicador total y barras por fuente.
Pastoreo	total_ranchos, epocas[]	Barras agrupadas y desglose por temporada.
Evolución	periodos[] con tipos	Barras porcentuales por temporada histórica.
Alerta	evolutionChange()	Calcula cambio de invasora entre primer y último periodo.

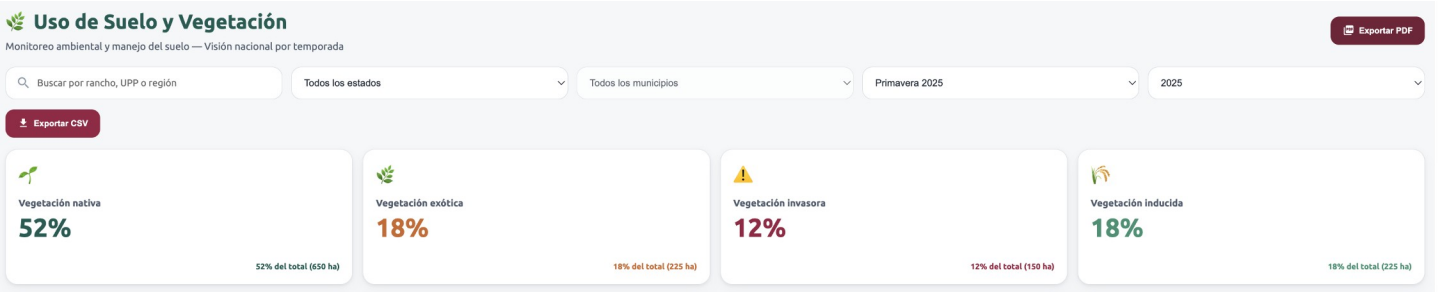


Figura 11. Disponibilidad de agua y tipos de pastoreo por época.

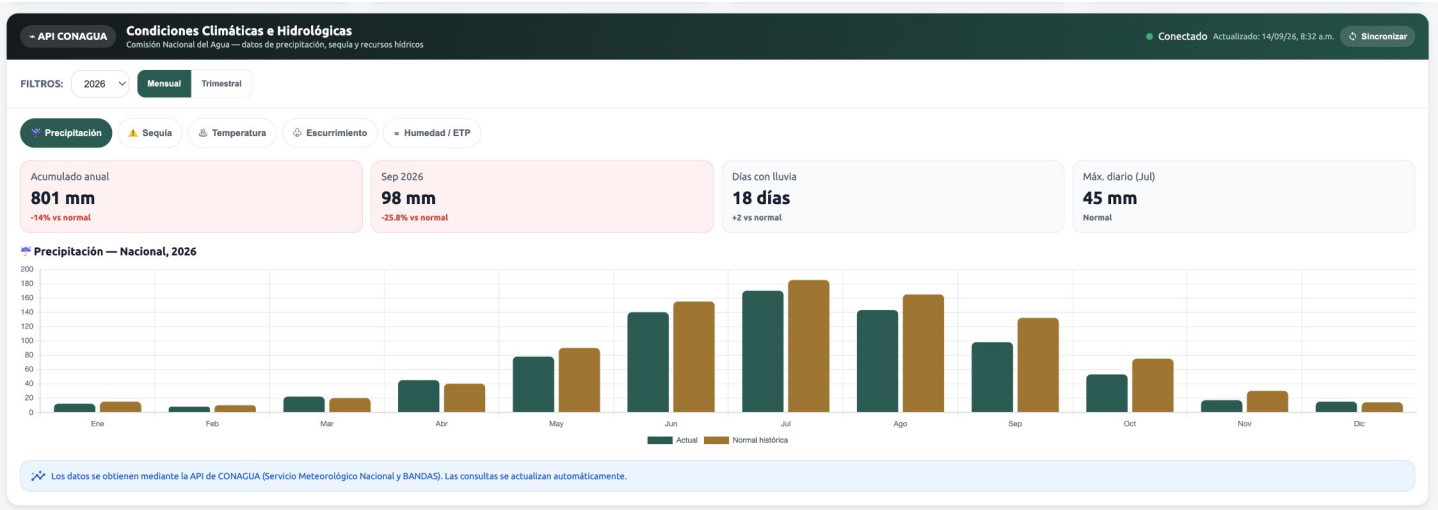


Figura 12. Evolución histórica de cobertura vegetal y alerta de tendencia.

13.4 Endpoints del módulo

Endpoint	Método	Uso
/api/uso-suelo/catalogos	GET	Catálogos de filtros y variables climáticas.
/api/uso-suelo/cobertura	GET	Cobertura actual por tipo vegetal.
/api/uso-suelo/cobertura/evolucion	GET	Serie histórica por temporada.
/api/uso-suelo/clima	GET	KPIs y serie climática.
/api/uso-suelo/agua	GET	Tomas de agua y fuentes.
/api/uso-suelo/pastoreo	GET	Tipos de pastoreo por época.
/api/uso-suelo/export	GET Blob	Descarga uso-suelo-{año}-{temporada}.csv.
/api/uso-suelo/clima/sincronizar	POST	Sincroniza datos climáticos.

/api/uso-suelo/cache/flush	POST	Path definido en configuración; no lo invoca este componente.

13.5 Integración climática con CONAGUA

La respuesta climática identifica fuente, conexión, fecha de actualización, región, variable, periodo, KPIs y serie. El botón Sincronizar llama POST /api/uso-suelo/clima/sincronizar; al terminar, recarga los cinco bloques de datos. La interfaz muestra Conectado o Sin conexión según climate.conectado.

Los datasets climáticos comparan actual contra normal histórica. formatValue combina valor y unidad; vs_normal_pct o diferencia_vs_normal se muestran como referencia de desviación.

13.6 Exportaciones y PDF

Exportar CSV llama /api/uso-suelo/export con filterParams y descarga uso-suelo-{selectedYear}-{selectedSeason}.csv. Exportar PDF envía #pdfContent a PdfExportService con nombre uso-suelo-vegetacion.pdf. El servicio común oculta filtros y controles durante la captura, conserva las gráficas renderizadas y divide el DOM en páginas A4 horizontal.

13.7 Base de datos lógica

El repositorio frontend no contiene SQL, migraciones ni entidades físicas. Los contratos observables permiten documentar estas entidades lógicas: cobertura vegetal, periodo/temporada, serie climática, rancho con agua, fuente de agua y manejo de pastoreo.

Entidad lógica	Campos observables	Relación funcional
Cobertura vegetal	tipo, ha, pct, superficie_total_ha	Se agrupa por temporada, año y región.
Periodo histórico	anio, temporada, etiqueta, tipos	Compara coberturas nativa, exótica, invasora e inducida.
Serie climática	periodo, etiqueta, actual, normal	Se filtra por variable, año, periodo y estado.
Fuente de agua	clave, etiqueta, ranchos, pct	Pertenece al resumen de disponibilidad de agua.
Manejo de pastoreo	época, tipo, ranchos, pct	Relaciona temporada con tipo de pastoreo.

Validar llaves, agregaciones, origen CONAGUA, periodicidad, caché y permisos con el backend. Esta sección describe contratos frontend, no el esquema físico.

13.8 Configuración y observaciones

```
npm install npm start # abrir https://bioparcela.encodingmx.com/admin/uso-suelo-vegetacion
```

La base del servicio está en usoSueloPath: https://giz-qa.encodingmx.com/api/uso-suelo.

Las llamadas usan isSecure=true y requieren sesión válida con Bearer.

selectedYear inicia en 2025; climateYear inicia en 2026. Son filtros independientes.

formatNumber usa locale en-US en este componente, distinto de otros módulos que usan es-MX.

usoSueloCacheFlush está configurado, pero el botón actual sincroniza clima mediante usoSueloClimaSincronizar.

14. Módulo Infraestructura del Rancho

El módulo Infraestructura del Rancho presenta el inventario físico, su condición funcional, el cercado y la necesidad de mejora a nivel nacional o por filtros geográficos. Se accede mediante /admin/infraestructura-rancho y se renderiza en el componente Infraestructura.

Resumen de infraestructura en estado bueno, regular y malo.

Inventario por tipo con cantidades, condición y porcentaje funcional.

Cercado del rancho por tipo, uso y superficie cubierta.

Detalle paginado por UPP y distribución de estado por tipo.

Comparación entre infraestructura funcional y necesidad reportada.

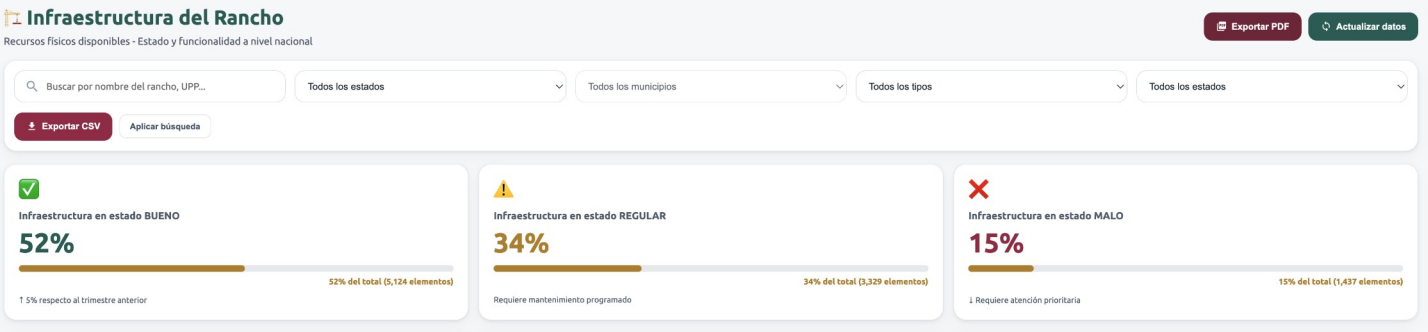


Figura 13. Filtros, estados e inventario de infraestructura.

14.1 Organización del código

```
src/app/pages/Infraestructura/infraestructura.component.ts      Lógica, contratos y carga de datos
src/app/pages/Infraestructura/infraestructura.html              Vista, tablas y controles
src/app/pages/Infraestructura/infraestructura.scss              Estilos del módulo
src/app/app.routes.ts                                           Ruta /admin/infraestructura-rancho
src/environments/environment.prod.ts                            URLs del servicio infraestructura
src/app/services/generic.service.ts                             HTTP y Authorization Bearer
src/app/services/pdf-export.service.ts                          Exportación DOM a PDF
```

El componente usa ChangeDetectionStrategy.OnPush, Chart.js/ng2-charts y PdfExportService. La vista combina tarjetas de estado, tablas de inventario, gráficas de cercado y tablas paginadas por UPP.

14.2 Flujo de carga y filtros

ngOnInit carga catálogos y llama loadData. loadData construye filterParams y ejecuta cinco peticiones con forkJoin: resumen, necesidad, cercado, desglose por UPP y cercado por UPP. Cada respuesta tolera error de forma independiente con catchError(() => of(null)).

Control	Parámetro	Efecto
Búsqueda	q	Filtra por nombre de rancho o UPP.
Estado	estado	Actualiza municipios y reinicia páginas.
Municipio	municipio	Restringe resultados territoriales.

Tipo	tipo	Filtra el tipo de infraestructura.
Condición	condicion	Filtra estado bueno, regular o malo.
Paginación	page, per_page	Consulta detalle UPP en páginas de cinco registros.

Al cambiar estado se limpian municipio y catálogo dependiente. Cualquier cambio de filtro reinicia las páginas de desglose UPP y cercado UPP a la primera página.

14.3 Funcionalidades y visualización

Panel	Contrato	Presentación
Resumen	total_elementos, estados[], inventario[]	Tarjetas bueno/regular/malo e inventario por tipo.
Necesidad	funcional, necesidad, brecha_pct	Barras de funcionalidad, necesidad y brecha.
Cercado	superficie cercada/sin cercar, tipos[]	Dona, tarjetas por cerco y superficie cubierta.
Desglose UPP	total, page, per_page, last_page, data[]	Tabla paginada con municipio y condición.
Cercado UPP	data[] con booleanos por tipo	Detalle de tipos de cerco usados por UPP.
Estado por tipo	bueno, regular, malo	Barras apiladas por elemento.

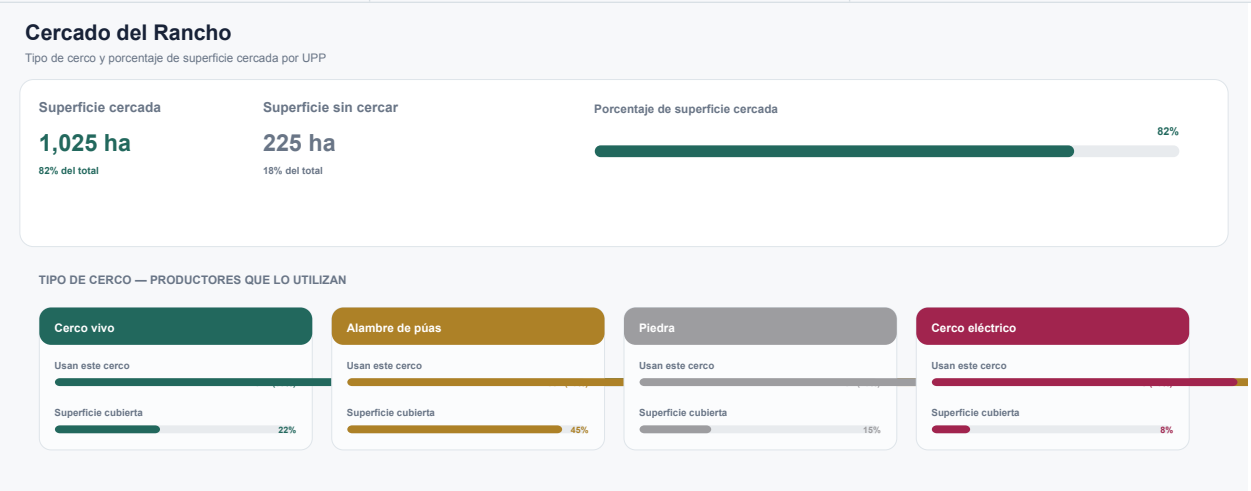


Figura 14. Cercado del rancho, tipos de cerco y superficie cubierta.

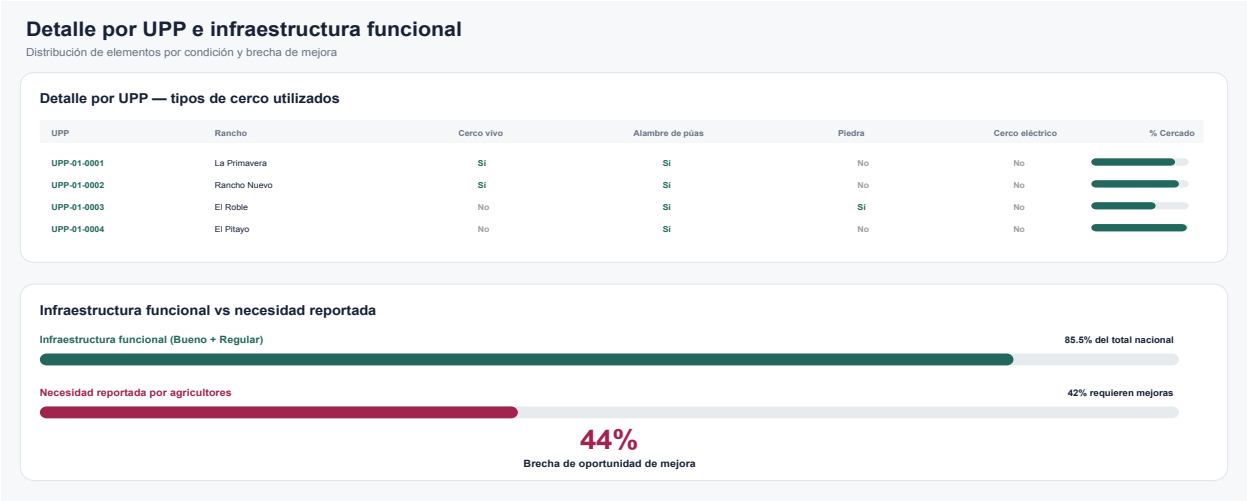


Figura 15. Detalle por UPP e infraestructura funcional frente a necesidad.

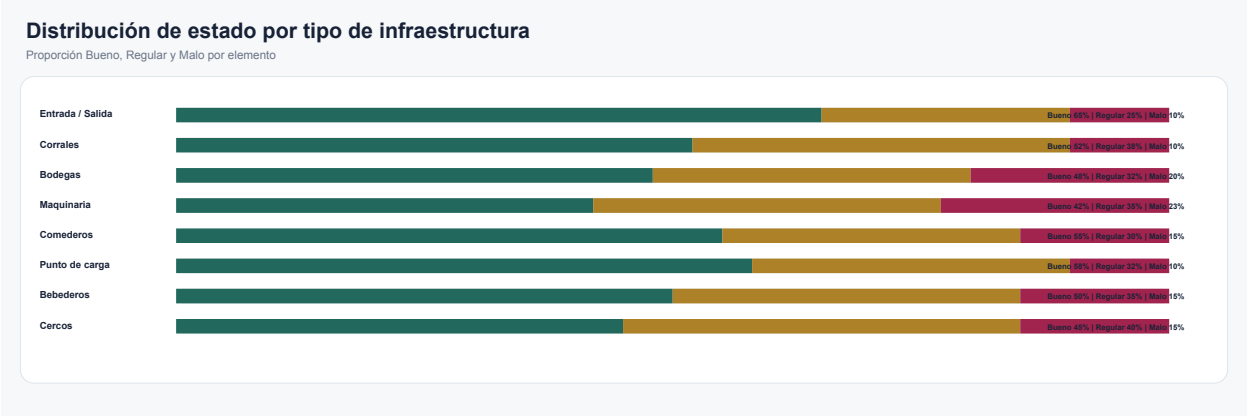


Figura 16. Distribución de estado por tipo de infraestructura.

14.4 Endpoints del módulo

Endpoint	Método	Uso
/api/infraestructura/catalogos	GET	Tipos, condiciones, cercos, estados y municipios.
/api/infraestructura/resumen	GET	Estados e inventario agregado.
/api/infraestructura/necesidad	GET	Funcionalidad, necesidad, porcentaje y monto estimado.
/api/infraestructura/cercado	GET	Superficie cercada y tipos de cerco.
/api/infraestructura/desglose-upp	GET	Detalle paginado de infraestructura por UPP.
/api/infraestructura/cercado/upp	GET	Detalle paginado de cercos utilizados por UPP.
/api/infraestructura/export	GET Blob	Descarga infraestructura-

		rancho.csv.
/api/infraestructura/cache/flush	POST	Actualiza la caché del módulo.

14.5 Cercado y paginación

El panel Cercado del Rancho construye una dona con superficie cercada y sin cercar. También construye una gráfica de barras horizontales apiladas con los productores que usan o no usan cada tipo de cerco.

setFenceUsage permite filtrar el detalle UPP por tipo de cerco y uso: todos, si o no. El filtro reinicia la página y envía tipo_cerco, uso, page y per_page. goToUppPage y goToFencePage limitan la página entre 1 y last_page.

14.6 Exportaciones y PDF

Exportar CSV llama /api/infraestructura/export con los filtros actuales y descarga infraestructura-rancho.csv. Exportar PDF entrega #pdfContent a PdfExportService con nombre infraestructura-rancho.pdf. Actualizar datos ejecuta POST /api/infraestructura/cache/flush y después recarga el módulo.

14.7 Base de datos lógica

El frontend no contiene SQL, migraciones ni entidades físicas. Los contratos observables permiten identificar estas entidades lógicas: infraestructura, condición, inventario UPP, cercado, tipo de cerco y necesidad de mejora.

Entidad lógica	Campos observables	Relación funcional
Elemento de infraestructura	tipo, etiqueta, total, bueno, regular, malo, funcional	Se agrupa por tipo y condición.
Inventario UPP	upp, rancho, municipio, total_elementos, estados	Relaciona unidad productiva con infraestructura.
Cercado	superficie cercada, sin cercar, pct, tipos[]	Resume cobertura de superficie por tipo de cerco.
Tipo de cerco	usan, no_usan, superficie_ha, superficie_pct	Se relaciona con productores y UPP.
Necesidad	elementos, ranchos, pct, inversion_estimada_mxn	Calcula brecha entre funcionalidad y mejora requerida.

Validar llaves, cardinalidades, criterios de condición, origen de inversión estimada, caché y permisos con el backend. Esta sección describe contratos frontend, no el esquema físico.

14.8 Configuración y observaciones

```
npm install npm start # abrir https://bioparcela.encodingmx.com/admin/infraestructura-rancho
```

- La base del servicio está en infraestructuraPath: https://giz-qa.encodingmx.com/api/infraestructura.
- Las peticiones usan isSecure=true y requieren sesión válida con Bearer.
- Los endpoints de detalle reciben page y per_page, con cinco registros por página en el componente.
- Los errores parciales no cancelan el resto de consultas; solo se muestra banner general si ninguna respuesta carga.

15. Módulo Sanidad, Vacunación y Certificaciones

El módulo Sanidad, Vacunación y Certificaciones presenta el estado sanitario del hato y la trazabilidad sanitaria. Se accede mediante /admin/sanidad-vacunacion y se renderiza en el componente SanidadVacunacion.

KPIs de cobertura de vacunación, certificaciones, protocolos y enfermedades bajo control.

Cartilla de vacunación y UPPs certificadas.

Registro de enfermedades por UPP, lote, animal y arete SINIIGA.

Protocolos de acceso, limpieza, sanitización y observaciones de auditoría.

Cobertura por enfermedad y recomendaciones para políticas públicas.

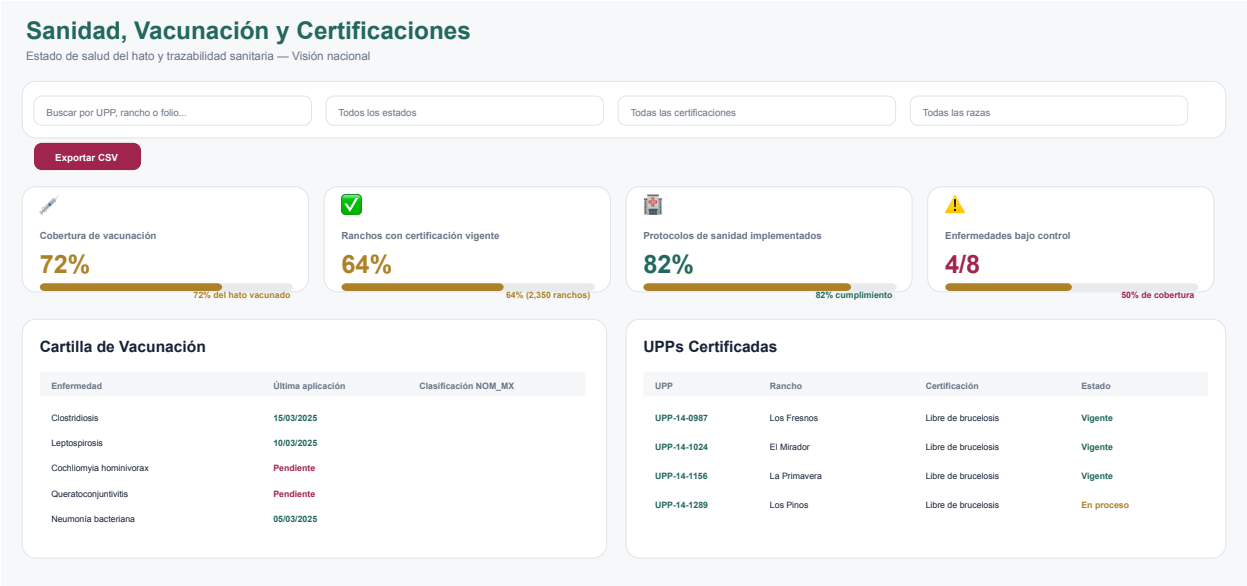


Figura 17. KPIs sanitarios, cartilla de vacunación y UPPs certificadas. Lámina basada en la captura de referencia compartida.

15.1 Organización del código

```
src/app/pages/SanidadVacunacion/sanidad-vacunacion.component.ts   Lógica, contratos y consultas
src/app/pages/SanidadVacunacion/sanidad-vacunacion.html             Vista, tablas y paneles
src/app/pages/SanidadVacunacion/sanidad-vacunacion.scss              Estilos del módulo
src/app/app.routes.ts                                                  Ruta /admin/sanidad-
vacunacion src/environments/environment.prod.ts                                 URLs del servicio
sanidad src/app/services/generic.service.ts                                   HTTP y Authorization
Bearer src/app/services/pdf-export.service.ts                               Exportación DOM a PDF
```

El componente usa ChangeDetectionStrategy.OnPush. No utiliza Chart.js en esta vista: presenta tarjetas, tablas, barras CSS, estados y recomendaciones a partir de respuestas HTTP.

15.2 Flujo de carga y filtros

ngOnInit carga catálogos y datos. loadCatalogs obtiene estados, certificaciones y razas. loadData construye filterParams y ejecuta siete consultas con forkJoin. Cada consulta tolera error individual con catchError(() => of(null)); si ninguna respuesta llega, se muestra el banner de error.

Control	Parámetro	Efecto
Búsqueda	q	Busca por UPP, rancho o folio.
Estado	estado	Filtra todos los bloques sanitarios.

Certificación	certificacion	Filtra UPPs certificadas y resultados relacionados.
Raza	raza	Restringe los registros sanitarios por raza.
Exportar CSV	filtros actuales	Descarga el padrón sanitario filtrado.
Exportar PDF	#pdfContent	Captura la vista completa con PdfExportService.

15.3 Funcionalidades y visualización

Panel	Contrato	Presentación
KPIs	kpis[]	Tarjetas con valor, unidad, progreso y detalle.
Cartilla	total, pendientes, items[]	Enfermedad, fecha, NOM_MX y folio de receta.
Certificaciones	total, items[]	UPP, rancho, certificación, fecha y estatus.
Enfermedades	total, items[], alerta	UPP, lote, arete SINIIGA, animal, fecha y estatus.
Protocolos	control_acceso_limpieza, sanitizacion, observaciones	Cumplimiento, productos y auditoría.
Cobertura	items[], promedio_nacional	Barras de cobertura por enfermedad.
Recomendaciones	items[]	Tarjetas de prioridad alta, oportunidad y alerta.

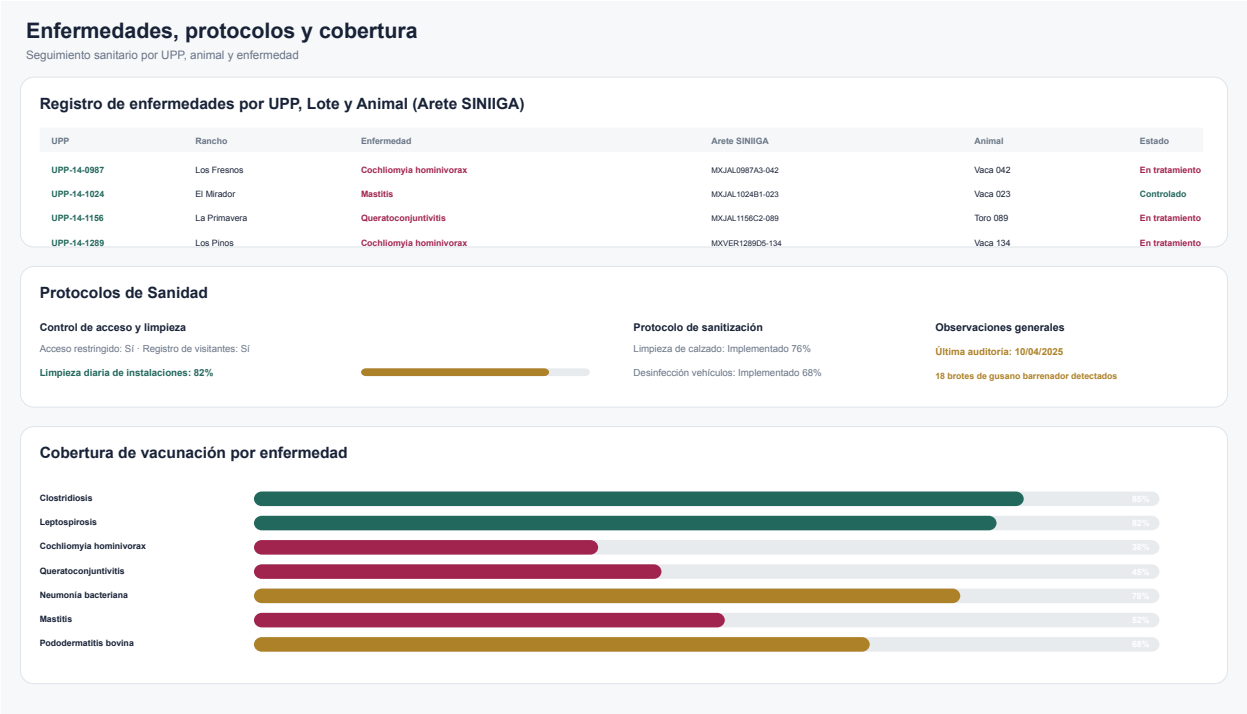


Figura 18. Enfermedades por UPP, protocolos de sanidad y cobertura de vacunación.

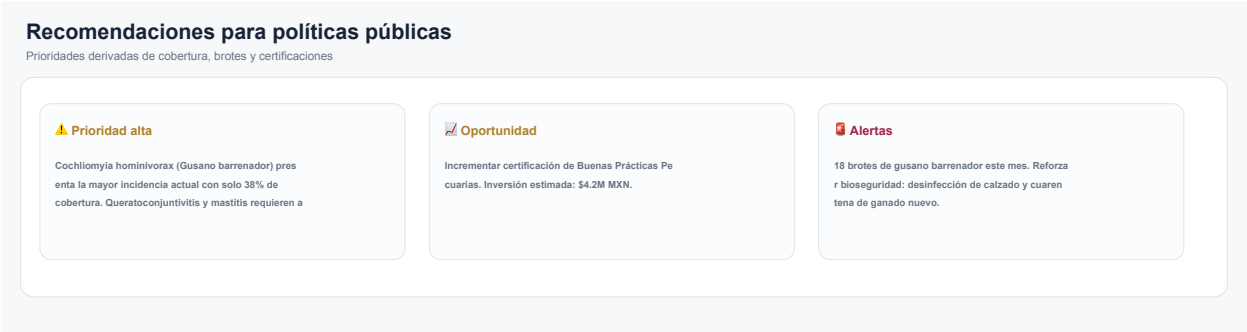


Figura 19. Recomendaciones derivadas de los indicadores sanitarios.

15.4 Endpoints del módulo

Endpoint	Método	Uso
/api/sanidad/catalogos	GET	Estados, certificaciones y razas.
/api/sanidad/resumen	GET	KPIs sanitarios.
/api/sanidad/cartilla-vacunacion	GET	Cartilla y pendientes de vacunación.
/api/sanidad/upp-certificadas	GET	UPPs y certificaciones vigentes.
/api/sanidad/enfermedades	GET	Casos por UPP, lote y animal.
/api/sanidad/protocolos	GET	Protocolos, limpieza, sanitización y auditoría.
/api/sanidad/cobertura-	GET	Cobertura de vacunación por

enfermedades		enfermedad.
/api/sanidad/recomendaciones	GET	Recomendaciones para políticas públicas.
/api/sanidad/export	GET Blob	Descarga sanidad-YYYYMMDD.csv.

15.5 Estados sanitarios y recomendaciones

statusClass agrupa estados aplicada, vigente y controlado como verdes; en_proceso como dorado; el resto como rojo. formatDate transforma fechas ISO con guiones al formato dd/mm/yyyy y usa Pendiente cuando no existe fecha.

Las recomendaciones se clasifican por tipo alta, oportunidad o alerta. recommendationIcon asigna un símbolo visual y la plantilla muestra título y texto entregados por el backend. El texto de recomendación debe considerarse salida de negocio, no lógica calculada localmente.

15.6 Exportaciones y PDF

Exportar CSV llama /api/sanidad/export con estado, certificacion, raza y q. El archivo se descarga como sanidad-{fecha actual YYYYMMDD}.csv. Exportar PDF envía #pdfContent a PdfExportService con nombre sanidad-vacunacion.pdf.

Diferencia frente a otros módulos: las peticiones de SanidadVacunacion no pasan explícitamente isSecure=true en GenericService. La autorización efectiva depende de la configuración de HttpClient/interceptores o del backend; revisar antes de producción.

15.7 Base de datos lógica

El frontend no contiene SQL, migraciones ni entidades físicas. Los contratos observables permiten identificar estas entidades lógicas: cartilla de vacunación, certificación UPP, enfermedad/animal, protocolo sanitario, cobertura y recomendación.

Entidad lógica	Campos observables	Relación funcional
Vacunación	enfermedad, ultima_aplicacion, NOM_MX, folio_receta	Registra aplicaciones y pendientes por enfermedad.
Certificación UPP	upp, rancho, certificacion, fecha, estatus	Relaciona unidad productiva con certificación sanitaria.
Caso de enfermedad	upp, lote, arete_siniiga, animal, fecha_deteccion, estatus	Relaciona enfermedad con lote y animal trazable.
Protocolo	acceso, visitantes, limpieza, sanitizacion, auditoria	Resume medidas de bioseguridad y revisión.
Cobertura	enfermedad, porcentaje, nivel	Agrega cobertura nacional por enfermedad.
Recomendación	clave, tipo, titulo, texto	Deriva acciones para políticas

		públicas.

Validar llaves, trazabilidad SINIIGA, normas NOM_MX, origen de casos, permisos y retención de expedientes con el backend. Esta sección describe contratos frontend, no el esquema físico.

15.8 Configuración y observaciones

```
npm install npm start # abrir https://bioparcela.encodingmx.com/admin/sanidad-vacunacion
```

La base del servicio está en sanidadPath: https://giz-qa.encodingmx.com/api/sanidad.

El módulo no implementa sincronización ni limpieza de caché en su componente actual.

Las listas vacías muestran mensajes específicos para los filtros seleccionados.

Los registros de enfermedad pueden activar una alerta operativa mediante diseases.alerta.